

Tracktention: Leveraging Point Tracking to Attend Videos Faster and Better

Sergei Gerasimov, Nazim Fadli

IP Paris

March 10, 2026

Based on the paper *Tracktention: Leveraging Point Tracking to Attend Videos Faster and Better* (Zihang Lai and Andrea Vedaldi, 2025).

- Motivation
- Idea of the paper
- Architecture overview
- Attentional Sampling
- Track Transformer
- Attentional Splatting
- Integration into backbone
- Point Tracker Pre-Processor
- Results

Motivation: Temporal Alignment in Videos

Standard Temporal Modeling

- *Implicit Methods*: 3D Conv and full spatio-temporal attention struggle with long-range dependencies and rapid motion due to computational complexity.
- *Explicit Flow-based Methods*: feeding optical flow to the network is fragile under occlusions and large, non-rigid displacements.

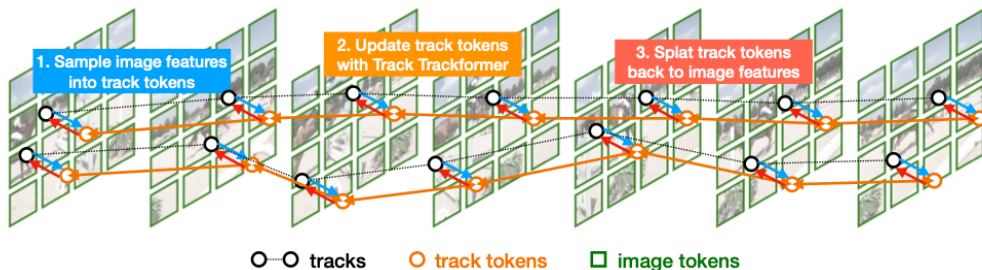
The Untapped Potential of Point Trackers

- Recent advances in point tracking networks provide highly robust, long-term motion trajectories.
- These "motion experts" handle occlusions and large displacements that break standard flow models.

Core Idea: The Tracktention Layer

- A **plug-and-play** transformer module that leverages the power of an off-the-shelf point tracker, providing a general mechanism to inject this knowledge into any video transformer. It enforces motion-aligned temporal consistency via sparse trajectory-guided attention.
- **3-Stage Mechanism:**
 - *1. Attentional Sampling:* Use an external point tracker to find moving objects across the video "tracks", then pool information from the video features right at those locations to create track tokens.
 - *2. Track Transformer (Update):* Processes the track tokens to propagate information along the tracks, making them more temporally consistent and smoothing out any irregularities.
 - *3. Attentional Splatting:* Symmetrically reverses the sampling process to map the updated track tokens back to onto the feature maps.
- It enhances temporal consistency in video transformers. Moreover, it can be added to single-image neural network architectures to instantly extend them into powerful, temporally-aware video models.

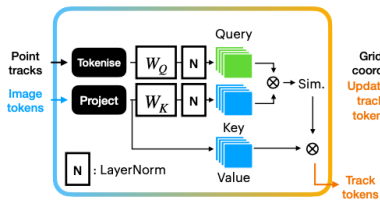
Core Idea: The Tracktention Layer



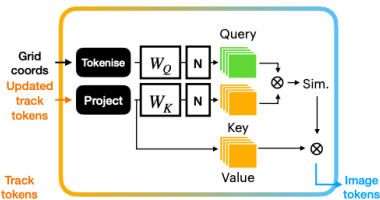
Overview of Tracktention: the 3-Stage Mechanism

Architecture overview

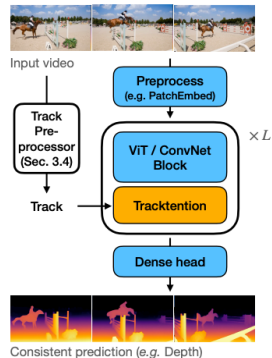
(A) Tracktention Architecture



(B) Attentional Sampling (Sec. 3.1)



(C) Attentional Splatting (Sec. 3.3)



(D) Integrating into existing models (Sec. 3.5)

Left: Tracktention architecture. Right: Integration in single-image neural network architectures to extend them to temporally-aware video models

Attentional Sampling: Setup and Notation

- The input feature map, where T is the number of time steps, H and W are the spatial dimensions, and D_f is the feature dimension. (flattened spatial grid): $F \in \mathbb{R}^{T \times HW \times D_f}$
- A Set of M tracked 2D point coordinates : $P \in \mathbb{R}^{T \times M \times 2}$
- Embed 2D points into feature space: $\mathcal{T}_t = \text{Embed}(P_t) \in \mathbb{R}^{M \times D_f}$
- We obtain one token per track and per frame, and arrange these tokens into a tensor : $\mathcal{T} \in \mathbb{R}^{T \times M \times D_f}$
- Each spatial token $j \in \{1, \dots, HW\}$ has known spatial location at time t (usually the center of the patch in image coordinates): $pos_{F_t}(j) \in \mathbb{R}^2$

$\mathcal{T}_{t,i}$ only Encodes spatial location of each track, No appearance information yet.

Cross-Attention Formulation

The track tokens attend to the the feature map. Project to queries and keys:

$$Q_t = \mathcal{T}_t W_Q \in \mathbb{R}^{M \times D_k} \quad K_t = F_t W_K \in \mathbb{R}^{HW \times D_k}$$

Values remain unprojected to preserve the original features during sampling:

$$V_t = F_t \in \mathbb{R}^{HW \times D_f}$$

Scaled dot-product logits (track to grid similarities):

$$L_t = \frac{Q_t K_t^\top}{\sqrt{D_k}} \in \mathbb{R}^{M \times HW}$$

$L_{t,i,j}$ how much track i matches grid token j at time t

Cross-Attention Formulation

Attention weights are computed as:

$$A_t = \text{softmax}\left(\frac{Q_t K_t^\top}{\sqrt{D_k}} + B_t\right)$$

To enforce spatial locality around each tracked point, a Gaussian spatial bias term is added:

$$B_{t,i,j} = -\frac{\|P_{t,i} - \text{pos}_{F_t}(j)\|^2}{2\sigma^2}$$

- Penalizes tokens far from the tracked location.
- σ controls the spatial bandwidth of attention.

Interpretation:

$$\text{softmax}(L + B) \propto \exp(L) \exp(B)$$

the bias is equivalent to multiplying attention scores by a Gaussian decays exponentially as the feature position deviates from the track position.

Attentional Sampling

Track features are obtained via attention:

$$S_t = A_t V_t = A_t F_t \in \mathbb{R}^{M \times D_f}$$

- Attentional sampling allows **learned, adaptive sampling**.

Rotary Position Encoding (RoPE) is applied to keys and values derived from the feature map:

- Encodes continuous spatial coordinates directly into feature representations.
- Makes attention aware of **relative spatial offsets** between track tokens and grid tokens.

Stabilization Details

- QK-normalization applied before attention.
- Multi-head attention (H heads) captures diverse spatial relations.

Track Transformer: Temporal Modeling Along Tracks

After Attentional Sampling:

$$S \in \mathbb{R}^{T \times M \times D_f}$$

Swap dimensions to treat each track independently:

$$\tilde{S} \in \mathbb{R}^{M \times T \times D_f}$$

Each track is now a temporal sequence of length T .

- Propagate information along motion trajectories (No interaction across tracks).
- Enforce temporal consistency per track.
- Smooth irregularities caused by occlusions or noise.

Track Transformer

For each track i , apply a 2-layer transformer encoder (treat M as batch dimension):

$$\tilde{\mathcal{S}}_i \in \mathbb{R}^{T \times D_f} \rightarrow \mathcal{T}'_i$$

Each layer consists of:

- **Multi-Head Self-Attention (over time):**

$$A = \text{softmax}\left(\frac{QK^\top}{\sqrt{D_k}}\right)$$

Each time step attends to all other time steps of the same track.

- **Position-wise Feed-Forward Network:** Nonlinear transformation applied independently at each time step.
- **Sinusoidal Temporal Positional Encoding:** Injected before the transformer to encode temporal order.

Output Updated track tokens:

$$\mathcal{T}' \in \mathbb{R}^{T \times M \times D_f}$$

Attentional Splatting: Reverse Cross-Attention

Map temporally refined track tokens $\mathcal{T}'$ back to the dense feature map F .

Reverse the roles of track and feature tokens in cross attention:

$$Q_t = F_t W_Q \quad K_t = \mathcal{T}'_t W_K \quad V_t = \mathcal{T}'_t$$

$$A'_t = \text{softmax} \left(\frac{Q_t K_t^\top}{\sqrt{D_k}} + B_t^\top \right) \in \mathbb{R}^{HW \times M}$$

- Image tokens query track tokens.
- Same Gaussian bias B_t is reused (transposed).
- QK-normalization and RoPE applied as in sampling.

Attentional Splatting: Write-Back to Feature Map

Aggregate refined track features:

$$U_t = A'_t[\mathcal{T}']_t \in \mathbb{R}^{HW \times D_f}$$

Final output projection (Tracktention output):

$$[\text{Tracktention}(F)]_t = W_{\text{out}} U_t$$

- Symmetric to Attentional Sampling.
- Ensures motion-consistent information flows back in a consistent manner.

Point Tracker Pre-Processor

To get motion trajectories

$$P \in \mathbb{R}^{T \times M \times 2}$$

before applying Tracktention.

Tracker: CoTracker3 (robust to large motion and occlusions)

- Trackers require seed coordinates (“queries”).
- We sample $M = 576$ queries **uniformly in the spatio-temporal video volume**.
- Each query is tracked forward and backward in time.

Why space-time random initialization?

- Grid initialization in frame 1 leads to coverage loss over time.
- Random space-time sampling maintains track coverage across frames.
- More robust to occlusions and late-entering objects.

Integration: From Image Models to Video Models

Key Application: Upgrade image-based networks to temporally-aware video models.

Tracktention can be inserted into existing backbones (ViT / ConvNet):

Block $\rightarrow$ Tracktention $\rightarrow$ Block $\rightarrow$...

Typically added after selected transformer or convolutional blocks.

Residual Integration

$$F' = F + \text{Tracktention}(F)$$

- Preserves original spatial representations.
- Injects motion-aware temporal consistency.

Stable Initialization

- Output projection initialized as $W_{\text{out}} = 0$.
- Ensures $F' = F$ at training onset.
- Allows gradual adaptation to temporal updates while retaining benefits of pre-trained weights.

Video Depth Estimation – Quantitative Results

Method	Type	#Params	Sintel (~50 frames)		Scannet (90 frames)		KITTI (110 frames)		Bonn (110 frames)		Average	
			AbsRel ↓	$\delta_{1.25}$ ↑	AbsRel ↓	$\delta_{1.25}$ ↑	AbsRel ↓	$\delta_{1.25}$ ↑	AbsRel ↓	$\delta_{1.25}$ ↑	AbsRel ↓	$\delta_{1.25}$ ↑
DUST3R [56]	Vid.	578M	0.628	0.393	0.194	0.694	0.292	0.456	0.250	0.588	0.341	0.533
NVDS [59]	Vid.	430M	0.408	0.483	0.187	0.677	0.253	0.588	0.167	0.766	0.254	0.629
ChronoDepth [49]	Vid.	1521M	0.587	0.486	0.159	0.783	0.167	0.759	0.100	0.911	0.253	0.735
DepthCrafter [20]	Vid.	1521M	0.343 [†]	0.673[†]	0.125	0.848	0.110	0.881	0.075	0.971	0.163	0.843
Marigold [27]	Im.	865M	0.532	0.515	0.166	0.769	0.149	0.796	0.091	0.931	0.235	0.753
DepthAny. [62] (*)	Im.	343M	0.325	0.564	0.130	0.838	0.142	0.803	0.078	0.939	0.169	0.786
DepthAny.-V2 [63]	Im.	343M	0.367	0.554	0.135	0.822	0.140	0.804	0.106	0.921	0.187	0.775
Ours	Vid.	140M	0.295	0.640	0.087	0.933	0.104	0.903	0.066	0.971	0.138	0.862

Table 1. **DepthCrafter [20] video depth benchmark.** Our model upgrades DepthAnything (*) to a video depth predictor, outperforming all baselines (image- or video-based) while having the smallest parameter count (140M). We use DepthAnything-Base (97M) as the base model and still outperform the DepthAnything-Large (343M) model. [†]Reproduced results confirmed by original authors.

Table 1: DepthCrafter video depth benchmark

Tracktention upgrades an image-based model to competitive video-level performance.

Video Depth Estimation – Qualitative Results

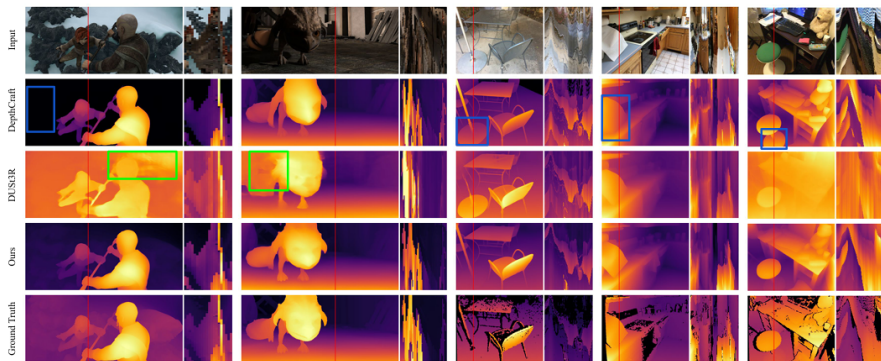
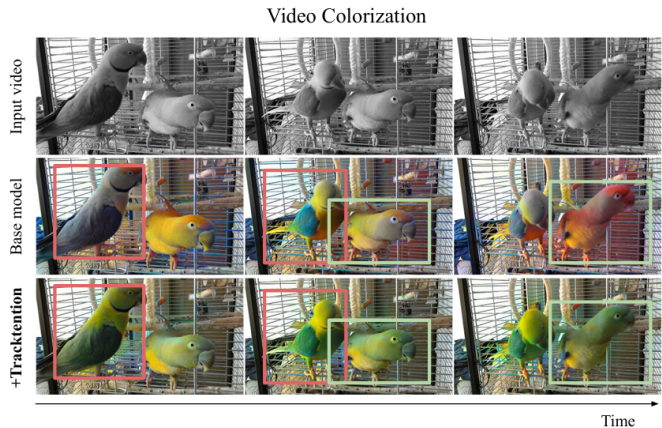


Figure 4. **Video depth prediction**, comparing Tracktention (+DepthAnything), DepthCrafter [20], and DUST3R [56]. We visualize a column of pixels (highlighted in red) over time to illustrate temporal variation. Our model shows stable, coherent depth estimation over time, while DepthCrafter exhibits significant errors in certain regions (blue box). DUST3R struggles with dynamic content (green box).

Figure 4: Depth prediction over time

Tracktention produces stable, coherent depth maps across frames.

Video Colorization – Qualitative Results



Tracktention Enforces motion-aligned consistency (same with Video Depth Prediction).

► Play video

Impact of Selective Tracks on Consistency

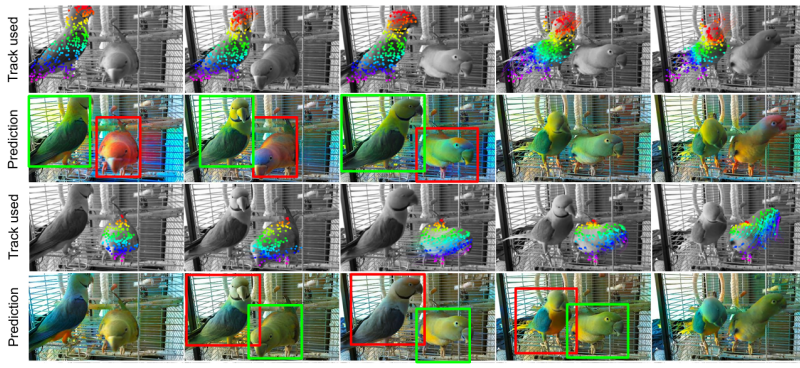


Figure 10: Effect of selective tracks

Confirms motion-guided selective attention behavior.

► Play video

Appendix: CoTracker3's Architecture Variants

- **Dual Model Approach:**

- **Online:** Sliding window (T' frames). Processes video sequentially. Memory-efficient, real-time, forward-only tracking.
- **Offline:** Processes entire video as one block. Bidirectional (forward & backward). Better for long-term occlusions.

- **Visual Representation:**

- Input: Video $I_1 \dots I_T$ + Query $Q(t_q, x_q, y_q)$.
- Output: Iterative updates for Trajectories (P_t), Visibility (V_t), and Confidence (C_t).

Appendix: CoTracker3's Feature Matching via 4D Correlation

- **Multi-Scale Feature Pyramid:**
 - 4 scales ($S = 4$), downsampled by $k = 4$.
 - Captures both fine details and global context.
- **4D Correlation Features:**
 - Extracts square neighborhoods ϕ_t around current estimates.
 - **Operation:** $\langle \phi_{t_q}^s, \phi_t^s \rangle$ — compares all vectors in query window vs search window.
 - **Efficiency:** High-dimensional correlations are projected via a simple **MLP** (simpler than LocoTrack).

Appendix: Iterative Updates & Supervision

- **Transformer Update Block (Ψ):**

- Input Token $G_t = [\eta(\text{displacements}), C_t, V_t, \text{Corr}_t]$.
- Updates: $P^{(m+1)} = P^{(m)} + \Delta P$ (repeated M times).
- Uses factorized time and group attention for efficiency.

- **Loss Functions:**

- **Tracks:** Weighted Huber loss (occluded points have 1/5 weight).
- **Confidence:** BCE loss where GT is 1 if error < 12 px.

- **Pseudo-labeling Tip:** During real-video training, C_t and V_t layers are frozen to prevent forgetting synthetic priors.

Appendix: Rotary Position Encoding (RoPE)

For a feature vector $f \in \mathbb{R}^d$ at spatial location (x, y) :

First $d/2$ channels encode the x coordinate.

For $i = 1, \dots, d/4$:

$$\text{RoPE}(f)[2i] = f[2i] \cos(\theta_i x) - f[2i + 1] \sin(\theta_i x)$$

$$\text{RoPE}(f)[2i + 1] = f[2i] \sin(\theta_i x) + f[2i + 1] \cos(\theta_i x)$$

$$\theta_i = 100^{-2(i-1)/(d/2)}$$

The second $d/2$ channels encode y analogously.

Key Property:

Rotations make dot-products depend on relative offsets: $Q(x_1)^\top K(x_2) = q^\top R(\theta(x_2 - x_1)) k$

Thus attention becomes relative-position aware.